SCAPMS Implementation Guide

1. System Scope & Core Rules

The Smart Clinic Appointment & Patient Management System (SCAPMS) is a C++ desktop GUI application for a university clinic to register patients, manage appointments and waiting queues, search and sort records, undo recent administrative edits, and generate summary reports. Active data resides in memory inside **ClinicDatabase** and persists to local **.txt**, **.csv**, or **.json** files via **FileHandler** without an external SQL server. Standard containers (**std::vector**, **std::queue**, **std::stack**) store the records, while all searching, sorting, and recursive algorithms must be written manually without built-in library search or sort functions.

2. Architecture & Object-Oriented Design

Source files are separated into logical header (**.h**) and implementation (**.cpp**) files across four directories and matching namespaces so GUI, controller, and data logic are never combined in a single class.

Directory	Required Classes	Responsibilities
/model	PatientRecord, GeneralPatient, EmergencyPatient, FollowUpPatient, Appointable, Appointment, ClinicDatabase	Defines the patient inheritance hierarchy, scheduling interface, appointment details, and the in-memory container store.
/controller	ClinicManager, SearchEngine, AppointmentController	Coordinates business rules, runtime polymorphism, manual search and sort routines, appointment updates, reports, and undo restoration.
/gui	MainWindow, PatientPanel, AppointmentPanel, AdminPanel, SearchPanel	Renders interactive forms, tables, menus, and dialogs; captures user and timer events; delegates work to controllers.
/utils	IDGenerator, FileHandler, Custom Exceptions	Generates sequential patient and appointment IDs, reads and writes local data files, and defines custom exception types.

Object-Oriented Hierarchy

- **PatientRecord (Abstract Base Class):** Contains Patient ID, Name, Age, Contact Information, associated **Appointment** objects, and at least one pure virtual function.
- **Concrete Subclasses: GeneralPatient**, **EmergencyPatient** (high-priority triage), and **FollowUpPatient** (returning visit), each providing behavior tailored to its patient type.
- **Appointable (Abstract Interface):** Declares pure virtual scheduling operations (**scheduleAppointment()** and **cancelAppointment()**) implemented by schedulable patient types.

- **Polymorphism & Composition:** Controllers process heterogeneous **PatientRecord** objects through base-class pointers or references at runtime. **ClinicDatabase** composes the patient collection, and each patient associates with one or more **Appointment** objects tracking date, service type, consultation notes, and status (Pending, Completed, or Cancelled).

3. Data Structures & Persistence

Data Structure	System Role	Behavior
std::vector	Master Patient Registry	Stores registered patients in memory and serves as the dataset for searching, sorting, and reports.
std::queue	Consultation Waiting Line	Manages waiting patients in FIFO order so the first arrival is normally served first.
std::stack	Administrative Undo History	Pushes state snapshots when deleting a patient, removing an appointment, or modifying a record so the latest action can be reversed.
Fixed-Size Array	Recent Patient Cache	Implements a 10-slot cache (e.g., **PatientRecord* recentPatients[10]**) for frequently accessed records.

4. Manual Algorithms & Reports

- **Searching (Two Manual + Recursion):** Implement **Linear Search** by Patient ID on unsorted data, **Binary Search** on sorted records, and a **Recursive Search** by patient name or category, selecting the algorithm based on whether the dataset is sorted.
- **Sorting (Two Manual):** Implement two algorithms from **Selection Sort**, **Insertion Sort**, **Merge Sort**, or **Quick Sort**. Provide dropdowns for *Sort Algorithm* and *Sort By* (supporting at least two fields among Patient Name, Patient ID, Age, or Appointment Date) plus a *SORT* button.
- **Simple Reports (At Least Three):** Generate views for (1) *Most Frequently Attended Patients* by completed appointments, (2) *Patients with Pending Appointments*, (3) *Patient Category Distribution* across General, Emergency, and Follow-Up groups, and (4) *Daily Appointment Summary* showing total, completed, pending, and cancelled counts.

5. GUI Design, Events & Exception Handling

- **Layouts & Panels:** Use at least two framework layout managers (e.g., **QVBoxLayout**, **QHBoxLayout**, **QGridLayout**, **QFormLayout**, or **QStackedLayout**) instead of manual positioning across the Patient Records, Appointments, Waiting Queue, Search & Sort, and Reports views.
- **Events & Advanced Features:** Wire button clicks, text changes, the *File* menu (New, Save, Load, Exit), and timer events (reminders, waiting alerts, dashboard refresh). Implement at least two additional enhancements: custom table rendering (Emergency → High Priority, General → Normal, Follow-Up → Scheduled), dynamic fields per patient type, file chooser dialogs, shortcuts (Ctrl+S, Ctrl+F, Ctrl+Q), and tooltips.

- **Exception Recovery:** Use **try/catch** blocks and at least one custom exception class to handle invalid patient data, duplicate IDs (*"Error: Patient ID already exists."*), invalid ages (*"Error: Patient age must be between 0 and 120."*), invalid appointments, file load failures, and invalid searches via error dialogs without crashing.

6. Implementation Order & Submission Deliverables

1. **Phase 1 – Scaffolding & Utilities:** Configure folders and namespaces, create custom exception classes, and build **IDGenerator**.
2. **Phase 2 – Domain Models:** Implement **Appointment**, **Appointable**, **PatientRecord**, and the three concrete patient subclasses.
3. **Phase 3 – Database & Persistence:** Build **ClinicDatabase** (vector, queue, stack, 10-slot array) and **FileHandler**.
4. **Phase 4 – Algorithms & Controllers:** Write manual search, sort, and recursive routines in **SearchEngine**, then build **AppointmentController** and **ClinicManager**.
5. **Phase 5 – GUI Wiring:** Build **MainWindow** and the four sub-panels using layout managers, connect events and timers, and attach validation popups.
6. **Phase 6 – Single-PDF Submission:** Combine labeled C++ source and runnable data files, 6 GUI screenshots (Dashboard, Registration, Appointments, Search/Sort, Queue, Reports), a UML class diagram (showing inheritance, abstract classes, polymorphism, and composition), and the 2–3 page 12-point project report into one PDF.